Smart Printer: Automated Paper Grading Machine

English | 中文

Resource

Demo Video
Website

Introduction

We have designed a machine for grading paper assignments, based on new large model technologies and product forms that achieved one-click scanning, markable grading, and data analysis.

Specifically, this system scans the whole assignment with one click and automatically print the grading results on the paper. All grading results are recorded in the cloud accessible at any time. In addition, the system has powerful data analysis capabilities that can statistically analyze the grading results and provide features such as error notebooks and weak knowledge point summaries for student users. The device can grade over 60 assignments per minute, with a per-page cost of approximately 5 cents and a grading accuracy exceeding 97%.

Compared to existing photo-based assignment grading software, our grading machine innovatively combines hardware and software, offering advantages far beyond traditional software:

Hardware: We communicate with conventional printer via standard printer interfaces to print the grading result. All printing tasks are completed by the system without any manual operation by teachers, achieving fully automated and traceable grading, greatly reducing the user's burden.

Software: we have adopted various methods to enhance the usability of this project:

- Model selection: We have based our system on a specially designed LLM adversarial interaction system, integrating RAG and MoE technologies, significantly improving model discrimination accuracy, coupled with user feedback, achieving accurate grading;
- Software performance: We have extensively used high-performance development methods, greatly improving processing speed and achieving fast batch grading.

Technical Innovations

The primary users of this project are:

- Teachers in primary and secondary schools
  Main application scenarios: regular assignment grading, school exam grading
- Students in primary and secondary schools
  Main application scenarios: repeated practice of test papers, error notebook usage

Our system integrates automatic test paper scanning, large model recognition, and grading functions, optimized by LLM (Large Language Model) to ultimately generate AI grading results. The backend uses Java, the frontend uses Vue.js/JavaScript, and testing uses Junit and Jest.

We utilized the following technologies:

- LLM Adversarial Interaction System
    - Single Model System Optimization
        - Hyperparameter tuning
        - GPT role setting
        - Retrieval-Augmented Generation (RAG)
    - Multi-Model System Optimization
        - Mixture of Experts (MoE)
            - Multi-agent discussion
            - Cross-validation
    - User Feedback
- User Request Acceleration and High Concurrency Processing
    - Kafka Message Queue
    - Redis Caching
    - Cloud Server Deployment
        - Auto-scaling
        - Elastic Load Balancing
        - Cloud Monitoring
- High-Performance Database Search and Scalability Optimization
    - Elastic Search
    - Database Optimization Techniques
        - Sharding
        - Replication
        - Partitioning
        - Write-Ahead Logging (WAL)
        - Auto-scaling

The following sections will detail these technologies:

LLM Adversarial Interaction System

To enhance the accuracy of the assignment grading system, we have designed a specialized model adversarial interaction system tailored for the scenario of primary and secondary school assignment grading. This system optimizes existing models, improving their judgment speed and accuracy. The system incorporates techniques such as hyperparameter tuning, GPT role setting, Retrieval-Augmented Generation (RAG), multi-agent discussion and cross-validation mechanisms inspired by Mixture of Experts (MoE) models. As a result, the problem-solving accuracy has increased by 8.2%, and model hallucinations has significantly decreased.

What is a Hallucination?

In large language models, "hallucination" refers to instances where the generated content does not align with real-world facts (factual hallucination) or is irrelevant to the user's input (fidelity hallucination).

For instance, in this project, when evaluating math problems, the model might produce content that is either irrelevant or contains factual errors:

- Factual hallucination, such as generating incorrect math computation results.
- Fidelity hallucination, such as explaining mathematical concepts instead of solving the problem.

These hallucinations mixed with other correct content generated by the model significantly impact the quality of the model's responses.

Single Model System Optimization

In the practical application of this project, the unoptimized large model has limitations in judgment accuracy due to its generality and lack of fine-tuning for specific student question banks.

The following methods were employed to optimize the single model system:

- **Hyperparameter Tuning**: During the fine-tuning phase of the pre-trained large model, techniques such as grid search and Bayesian optimization were used to determine the optimal combination of hyperparameters (e.g., Learning Rate, Batch Size, and Regularization Parameters) to improve model performance. Here are a few hyperparameters adjusted for this project:
    - **Learning Rate**: Controls the step size of parameter updates. A lower learning rate ensures more stable convergence but requires longer training time; a higher learning rate is the opposite. This project's learning rate is generally set around the 1e-6 level.
    - **Epochs**: Controls the number of times the training dataset is fully traversed. More epochs allow for more thorough learning but may lead to overfitting. In this project, epochs are adjusted to around 500.
    - **Temperature**: Adjusts the randomness in the final word selection from the probability distribution to control the randomness of the content generated by the large model. A higher temperature value results in a more even probability distribution, greater randomness, and more creative and diverse content, but also increases the likelihood of generating irrelevant or incorrect answers. In this project, the temperature is generally set to a lower value, currently at 0.3 to ensure the generated content does not deviate too much from the question.

  Additionally, there are many other hyperparameters that are not listed here. To enhance the judgment accuracy of the large model in this project while ensuring response speed is not compromised, these hyperparameters need to be adjusted and combined to achieve overall optimal values. For example, using a small sample of math problems with confirmed answers for initial learning and fine-tuning hyperparameters based on the results. We regulate the number of iterations by using hyperparameter optimization techniques such as Bayesian optimization to first identify an approximate value for this parameter. Then, through experimentation, we determine the optimal value so that the large model makes as many iterations as possible on the dataset to ensure adequate learning without overfitting. Next, we adjust the temperature value to enhance the accuracy of the model's generated content. After determining the optimal values for several parameters, we combine the hyperparameters and conduct experiments to find the optimal combination ensuring the single model achieves the best results in practical applications (grading a large number of math problems).

- **GPT Role Setting**: By explicitly specifying the model's role during training, such as chat, assistant, or teacher, the model can be more targeted and adaptable in specific scenarios, thereby enhancing its performance in particular tasks. In this project, we generally choose the teacher role, which is more aligned with the needs of the target users and offers advantages in judgment, improving the model's ability to handle factual issues.
- **RAG (Retrieval-Augmented Generation)**: By incorporating relevant background knowledge during model content generation, the model can generate more accurate and relevant content, reducing the likelihood of hallucinations. Based on this, to further improve the judgment accuracy of a single model, we adopted the RAG technique.. The technique vectorizes data from a knowledge base and stores it in a database. When the user query is made, related knowledge is retrieved, and answers are generated based on this knowledge. In this project, we prepared a specialized knowledge base covering primary and secondary school curriculum content and exam question banks. Using these knowledge bases to generate answers can effectively improve the decision-making accuracy of the single model.

Performance Optimization Using Multi-Model Systems

In this project, although the accuracy of a single model combined with RAG technology has improved, there is still room for improvement. Preliminary validation results showed that the accuracy of answers generated by a single model using RAG was slightly above 80%. To further improve accuracy, we have introduced a multi-model discussion mechanism inspired by students discussing and cross-checking answers: multiple models discuss and cross-check each other’s results to enhance the reliability of the final answer. This technique includes two key points:

Multi-Agent Discussion

The system is deployed via AWS cloud services. EC2 instances are used for deploying main application code, computation, and running services; S3 is used for storing static resources and data; Amazon RDS is used for database hosting in the cloud, handling tasks such as backup, recovery, fault detection, and repair, ensuring high fault tolerance and disaster recovery capability; Lambda is used for serverless computing, enabling automated tasks and event-driven function execution.

The following process is designed to implement multi-agent discussion:

- **Request Reception**: The primary modules of this project are deployed on AWS EC2 instances. These instances, which receive client requests, parse task content, and execute corresponding processing logic.
- **Multithreading Processing and Request Distribution**: Using Java-based multithreading technology for parallel processing, we send requests to multiple models simultaneously, allowing multiple models to handle the same problem concurrently, thereby improving efficiency.
  For example, after a math problem is uploaded and parsed, multiple threads are opened, each corresponding to an independent model dialogue. The math problem content is then distributed to multiple threads, allowing large models to process the problem simultaneously. In practical use, if a large model's maximum response time for a question is 5 seconds, sequential processing means querying each model one by one, with the total waiting time being the sum of all models' response times, far exceeding acceptable limits. By using parallel processing, the maximum wait time is only 5 seconds.
  After recognition, the data processing module on AWS EC2 integrates the recognition results from each model, deriving more accurate and comprehensive conclusions.

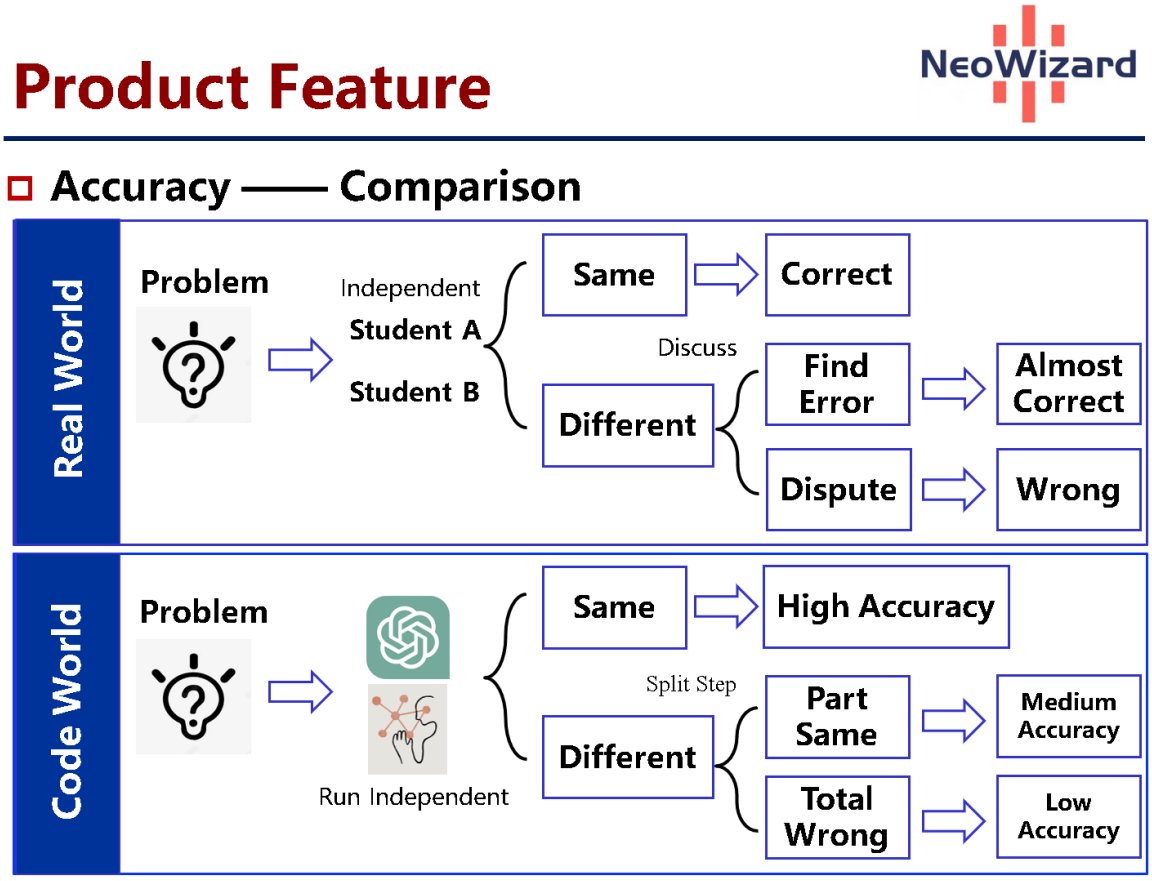
In this process, thread pools are used for thread management and reuse, described as follows:

In this project, thread pool technology is employed to achieve higher parallel processing efficiency. Excessive threads increase scheduling overhead, affecting cache locality and overall performance. Therefore, thread creation and destruction need to be reasonably managed. By creating a thread pool in advance, tasks can be processed in parallel using threads from the pool, with specific logic used to manage and reuse threads to improve efficiency. In this project, the Executor framework is used to create a fixed-size thread pool, providing thread reuse and scheduling functionality.

- **Result Integration**: The data processing module on AWS EC2 coordinates the responses from different models, undergoes a cross-validation process, and finally derives a consistent answer.

During the multi-agent discussion, if the results from different models are consistent after semantic consistency checks and logical consistency analysis, the answer is deemed correct. If inconsistent, a cross-validation step (which will be introduced below) is performed to compare and judge to avoid errors. The following diagram illustrates the specific workflow.

Currently, the large models we use include Llama-3-70b-Groq, Claude-3.5, the multimodal large model GPT-4o, Gemini, and others. By introducing multimodal large models, we ensure that the system can effectively handle image-based questions and drawing problems, while also leveraging the advantages of large language models in processing natural language. Introducing these models not only combines the strengths of each model but also minimizes the probability of errors through multi-agent discussion. Additionally, we can replace or add more powerful models in the future.



Cross-Validation

The primary goal of introducing RAG is to acquire more knowledge to generate more accurate responses. Although it can reduce hallucinations to some extent, its effect is still limited.

Hallucinations in generated content are relatively rare and specific, with each model generating different hallucinations. Thus, by comparing and querying the generated results of multiple models, hallucinations can be effectively reduced. Based on this characteristic, we introduce a cross-validation method to maximize the elimination of hallucinations. The main steps of this method are as follows:

- **Initial Generation and Summarization**: The server aggregates the questions and the responses from multiple models to determine if there are any hallucinations. Judgment techniques include semantic consistency checks, factual verification (e.g., numerical accuracy), and logical consistency analysis.
- **Secondary Query and Scoring**: Based on the initial aggregation, the server opens a new thread for each model. Through these new threads, the server sends the aggregated answer back to each model for a secondary challenge. Queries include:
    - Scoring these responses.
        - Attempting to generate better responses based on all acquired information.
- **Answer Integration**: Based on scoring and generation results, hallucinations are removed and replaced with correct content. If the scoring results significantly favor one answer, that answer is adopted, indicating model consensus; if indeterminate, the querying process is repeated.

This process is conducted for up to three rounds to ensure hallucinations are minimized and accuracy is maximized within a short processing time.

User Feedback

To further improve the accuracy of final judgment results, this project incorporates user feedback. The final answers generated after single model and multi-model discussions are stored in the database, marked as AI-generated, and await verification. If the grading teacher does not raise any objections to the answer after the exam explanation, it is confirmed as correct and marked accordingly; otherwise, it is regenerated or manually graded.

By adding this step, we ensure that the grading data stored in this project is correct. Even in rare cases where the large model system makes an error, the first grading teacher's error report can promptly correct it, preventing misguidance in subsequent grading.

Summary

By fine-tuning hyperparameters, setting GPT roles in single models and employing multi-agent discussions in multi-model systems, we have optimized the grading system's performance. For single models, RAG technology is used, and for multi-model systems, a cross-validation strategy is adopted to minimize hallucinations, significantly improving the accuracy of the AI grading system. Finally, user feedback is incorporated to ensure the grading results are foolproof. In practical applications, the AI judgment system has improved problem-solving accuracy by 8.2%. Due to the user feedback mechanism, the stored grading results in the database are 100% accurate, ensuring the reliability and consistency of the grading results.

User Request Acceleration and High Concurrency Handling

As the project progresses, the volume of data processed by this system will expand rapidly. For example, statistics from currently deployed schools show that each class has about 40 math assignments daily, resulting in approximately 5000 assignments per semester. When considering all classes, multiple subjects in the same class, and mid-term and final exams, the total grading volume for a single school reaches a significant level, with an even more staggering number of questions involved. Therefore, this application scenario is quite a test of the database calling and

searching ability. Additionally, teachers may use this system to grade assignments at any time, and students may frequently access past questions for review, especially before mid-term and final exams, which demand high concurrency capabilities from the project.

In order to maximize the system's speed and efficiency to improve usability and user satisfaction, we have implemented various acceleration techniques at the software level by using high-performance development knowledge in different phases of the search. We use Kafka message queues to handle query peaks, use Redis to cache recent queries and answers for quick LLM responses, and utilize Elasticsearch for rapid search and indexing of the question database. Finally, we have also optimized the database to enhance its usability further.

Kafka Message Queue

Given the project's nature, search requests are usually not excessively high, as different teachers are unlikely to use the system simultaneously for grading. However, during peak periods (e.g., final exams), search requests may surge in a short time. To address this, we use Kafka queues, a high-throughput distributed messaging queue that plays a critical role in handling a large number of concurrent requests during the grading process. It can effectively balance server load during these peak periods.

When a user submits a search request, it first passes through the Amazon API Gateway, which maintains and monitors APIs and provides authentication services, receiving user requests and forwarding them to servers deployed on AWS cloud services. The backend server formats the request data and sends it to the Kafka queue through asynchronous calls. Kafka partitions the requests and assigns them to multiple consumer nodes for processing, which can be distributed computing services. Kafka publishes these grading tasks as messages to the corresponding Topic, where each grading node acts as a consumer and retrieves the assignments from Kafka for processing. The processing results are returned in a similar manner.

During processing, asynchronous calls help avoid synchronous blocking, enhancing request processing concurrency. Kafka's partitioning and consumer node mechanism enable parallel processing, reducing each request's processing time, thus speeding up the overall processing. Additionally, during peak periods, Kafka can balance the load, avoiding single-point bottlenecks and ensuring system stability and efficient operation. Overall, this approach not only accelerates search request processing but also ensures system stability during peak periods.

In addition, Kafka allows us to monitor the working status of each node in real time and analyze the progress of task processing. Moreover, Kafka's built-in message backup and fault tolerance mechanisms enable quick data recovery in the event of node failure or network fluctuations, ensuring the continuity and integrity of the grading tasks.

Here is a practical application example of Kafka queues in this project:

> Suppose during final exam grading, math teachers collectively upload students' exams for grading, resulting in a large number of concurrent queries. At a certain moment, three math teachers simultaneously upload 200 assignments for grading. These requests first pass through the API Gateway to the backend server, which formats the requests and sends them to the Kafka message queue. Assume we have three processing nodes. Based on monitoring usage, I/O traffic and other factors, Node 1 has a load significantly higher than other nodes. Kafka distributes these grading requests mainly to Nodes 2 and 3, with each node handling a portion of the grading tasks, which achieves load balancing and avoids overloading a single node. After processing, nodes send the grading requests back to the Kafka message queue, which then returns them to the server. The server then sends the information back to the frontend, allowing teachers to see the results they need.

This method ensures that even under high concurrency conditions, the system can respond to each user request stably and efficiently. Currently, our servers can handle 500 QPS, and we plan to further improve the system architecture and expand processing capacity if demand increases in the future.

Redis Cache

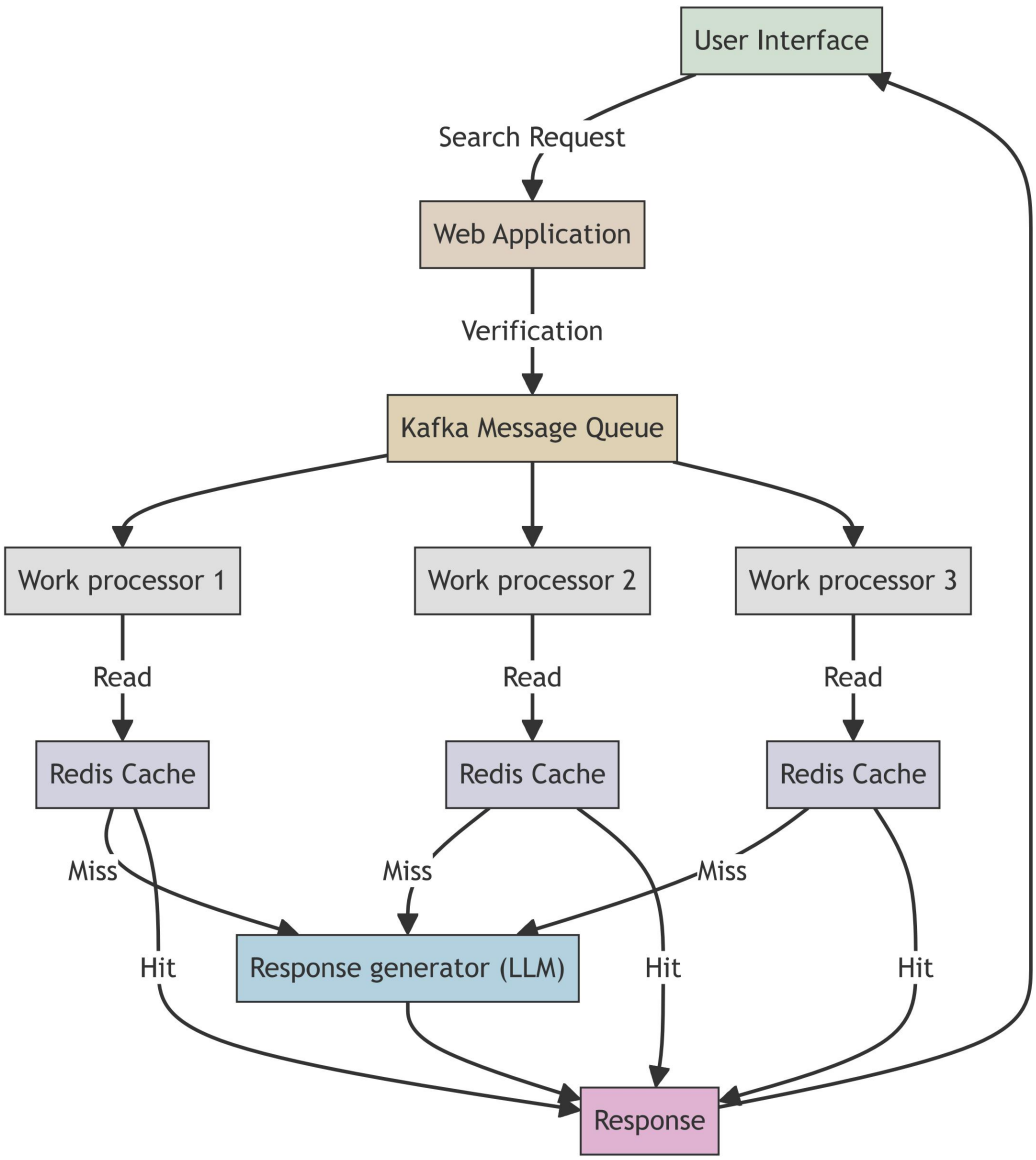
While Kafka primarily addresses high concurrency issues, we introduced Redis technology to further enhance system response speed. Redis is an in-memory cache system, and we use a capacity-based cache eviction policy to store recent queries and responses for quick retrieval.

Based on usage statistics from deployed schools, the same grade often uses the same set of exams for assignments and tests. Therefore, different teachers in the same school are likely to call the same questions during grading. Additionally, recently called questions are often new and more likely to be used by teachers from other schools in the same period. Given curriculum reforms and textbook revisions, older questions are less likely to be used. Thus, we use the LRU (Least Recently Used) algorithm to evict the oldest data and retain more recent data.

Upon receiving a query, the system first checks the Redis cache. If the cache is hit, the cached response is immediately returned. This step is entirely based on single-threaded in-memory operations, making it very fast and significantly reducing response time compared to reading data from the database. If no match is found, the server sends the query to the backend database and caches the result in Redis for future use.

When Kafka and Redis work together in the system, Kafka handles high-concurrency message queues, while Redis provides fast caching services. When the system receives a query request from the Kafka queue and completes data processing, the backend server stores the result in Redis. Thus, the next identical query can directly retrieve the answer from Redis cache without re-querying the Kafka queue or database.

By extensively using this caching mechanism in the model, we achieve high availability, significantly reduce the backend server load, and optimize overall system performance.



Cloud Server Deployment

To further enhance system reliability and scalability, we adopted the following technologies on AWS:

- **Auto Scaling**: Automatically increases or decreases the number of processing nodes based on load, ensuring sufficient resources to handle requests during peak periods.
- **Elastic Load Balancer (ELB)**: Distributes traffic across multiple backend instances, providing high availability and failover capabilities.
- **CloudWatch**: Monitors system performance and operational status in real-time.

High-Performance Database Search and Scalability Optimization

In this project, users need to view historical questions. Teachers often call recent grading questions, but the project also provides students with historical question review and error notebook services. Students may frequently query past mistakes before exams, leading to frequent database queries. Database search and indexing capabilities are crucial to ensure users can quickly and accurately retrieve the needed questions. To enhance these capabilities, we introduced Elasticsearch, a dedicated full-text search engine based on the open-source Lucene. Additionally, we optimized the MySQL database through sharding, replication, partitioning, write-ahead logging, and auto-scaling to enhance scalability and durability.

Elasticsearch

As previously mentioned, the database needs to accommodate a large volume of questions from different subjects. Traditional database search methods face the following issues:

- **Efficiency**
  SQL database queries like `name LIKE` use table scans, which are less efficient than indexing, leading to slower searches. While MySQL offers indexed queries, forward indexing can still be slow with large data volumes, which is evident in this project's database.
- **Fuzzy Query**
  Fuzzy queries return large amounts of data, causing users to spend extra time searching through results. This is a significant issue in this project, as similar questions often have similar stems.
- **Format Matching**
  Traditional text matching methods may fail to retrieve results due to format differences. For example, math formulas stored in LaTeX format may not match queries using different representations during retrieval.

Elasticsearch addresses these issues through distributed architecture and parallel processing. It uses index name splitting and multi-shard indexing techniques. Cross-index queries can use direct (specifying index names), fuzzy (using wildcards), and computational (specifying indexes via expressions) methods, greatly enhancing the likelihood of retrieving highly relevant content.

- **Efficiency**: Elasticsearch stores data using the inverted index, which finds documents or information IDs that contain keywords based on the content of the documents. It maps keywords to document IDs, enabling quicker content searches than forward indexing that stores data by ID order, not to mention table scans without indexes, etc.
- **Fuzzy Queries**: Elasticsearch supports various query methods, including direct, fuzzy, and computational index methods, maximizing the chances of finding highly relevant content and avoiding cluttered information. This fuzzy search can also address formula format conversion issues, allowing searches to find needed data without exact matches, which traditional full-text searches cannot achieve.

Database Optimization Techniques

We implemented the following methods to enhance database usability:

- **Sharding**
  Data is sharded across multiple nodes, with each node handling a portion of queries, ensuring load balancing and high availability. Sharding also allows data replication across multiple database instances, enhancing data availability and disaster recovery.
- **Replication**
  By creating multiple replicas of the database across different nodes, we enhance data

reliability and disaster recovery capabilities.

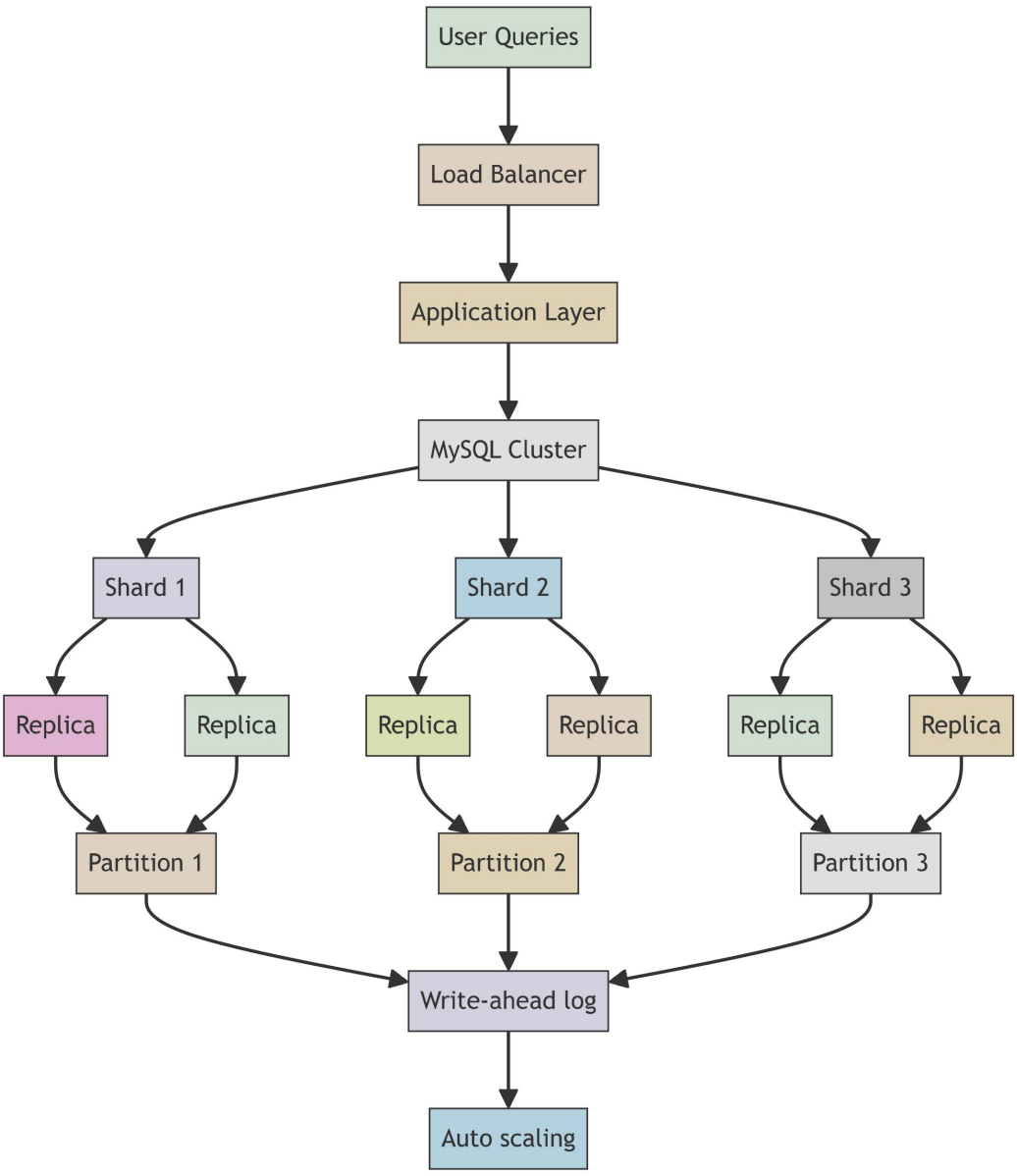
- **Partitioning**
  Data is partitioned based on range, list, or hash rules into different physical storage areas, reducing data volume in a single physical storage area and optimizing query efficiency and data management.
- **Write-Ahead Logging (WAL)**
  During transaction execution, all modifications are first recorded in the WAL before being applied to the database, ensuring data consistency and durability.
- **Auto Scaling**
  By monitoring database load, the system automatically adjusts the number of database instances, ensuring sufficient resources during high load and saving resources during low load.

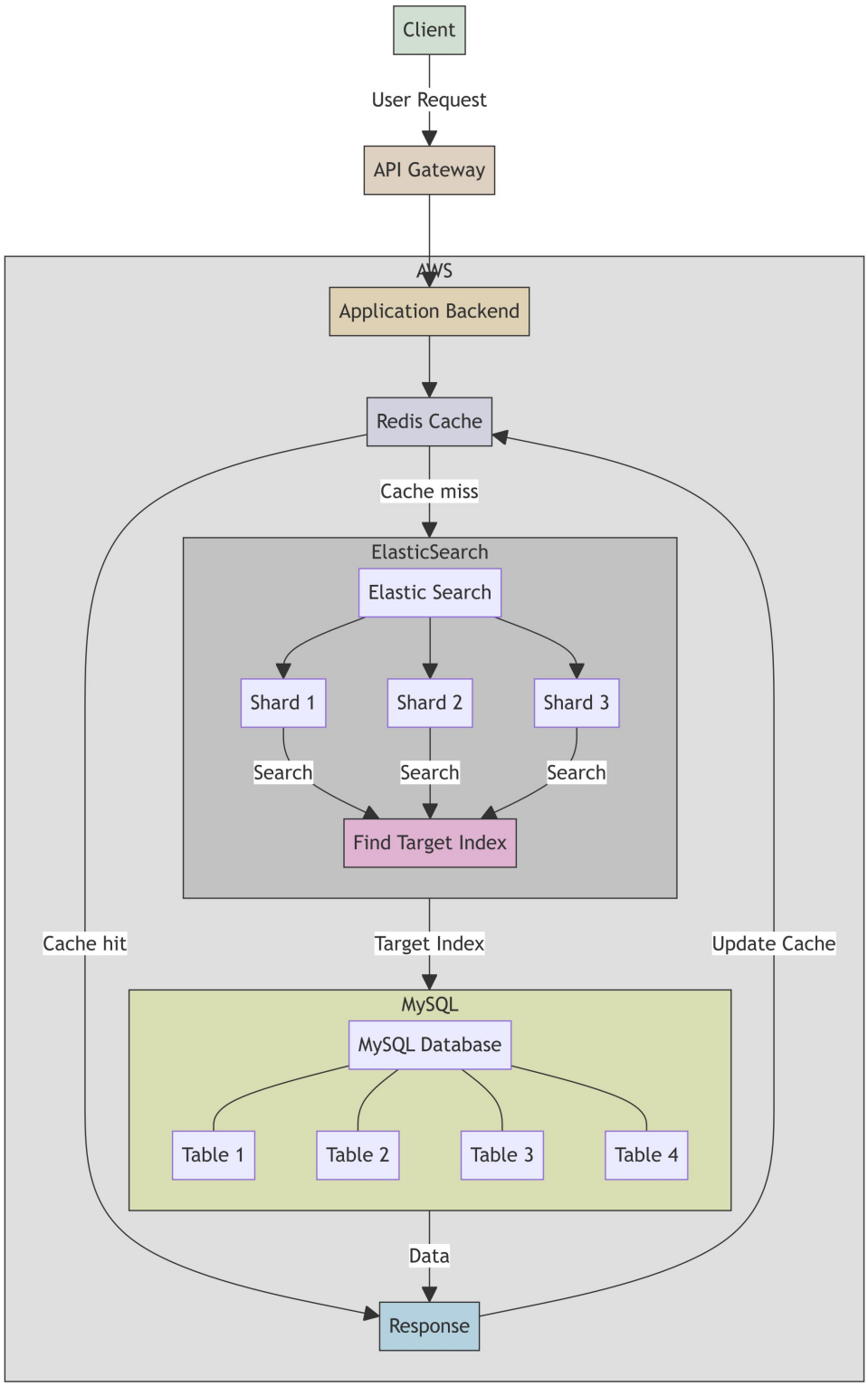
Summary

In this project, database operations generally follow these steps:

- **Data Input and Indexing**
  Data in the database is indexed by Elasticsearch upon generation or update. The indexing process splits data into multiple shards and stores them on different nodes for efficient distributed processing.
- **Query Processing**
  When a user submits a question query, the server receives and first writes it to the Kafka message queue. Kafka partitions the request and assigns it to multiple consumer nodes for processing. During processing, the system first checks the Redis cache. If the cache is not hit, the query is sent to Elasticsearch. Elasticsearch processes the query using its efficient indexing and sharding mechanisms and returns the result. These mechanisms include:
  - **Inverted Indexing**: Stores document content using an inverted index structure, allowing quick searches for documents containing specific terms. Compared to forward indexes, inverted indexes significantly enhance search speed and efficiency.
  - **Index Sharding**: Each shard is an independent inverted index, allowing data distribution and processing across multiple nodes, achieving horizontal scalability. Each shard can independently handle query requests, sharing the load and avoiding single-point bottlenecks, enhancing the system's concurrent processing capacity.
  - **Replica Shards**: Copies of primary shards stored on different nodes, increasing data reliability and availability. This increases the likelihood of finding the relevant content during indexing and retrieval across different nodes. Replica shards not only improve data reliability and availability but also provide additional parallel processing capabilities during queries, further enhancing query speed.
- **Result Return and Caching**
  Query results are returned via the Kafka queue, and the system caches the results in Redis for faster response to future identical queries.
- **Data Consistency**
  The backend server ensures consistency between Elasticsearch indexes and the primary database. All data reads are recorded in the WAL and the Elasticsearch index is updated synchronously.

Here is a general schematic and structural diagram of database usage in this project:


User Queries
Load Balancer
Application Layer
MySQL Cluster
Shard 1
Shard 2
Shard 3
Replica
Replica
Replica
Replica
Replica
Replica
Partition 1
Partition 2
Partition 3
Write-ahead log
Auto scaling



Software and Hardware Integration

The innovation of this project lies not only in the adoption of cutting-edge technologies in software development but also in the integration of software and hardware.

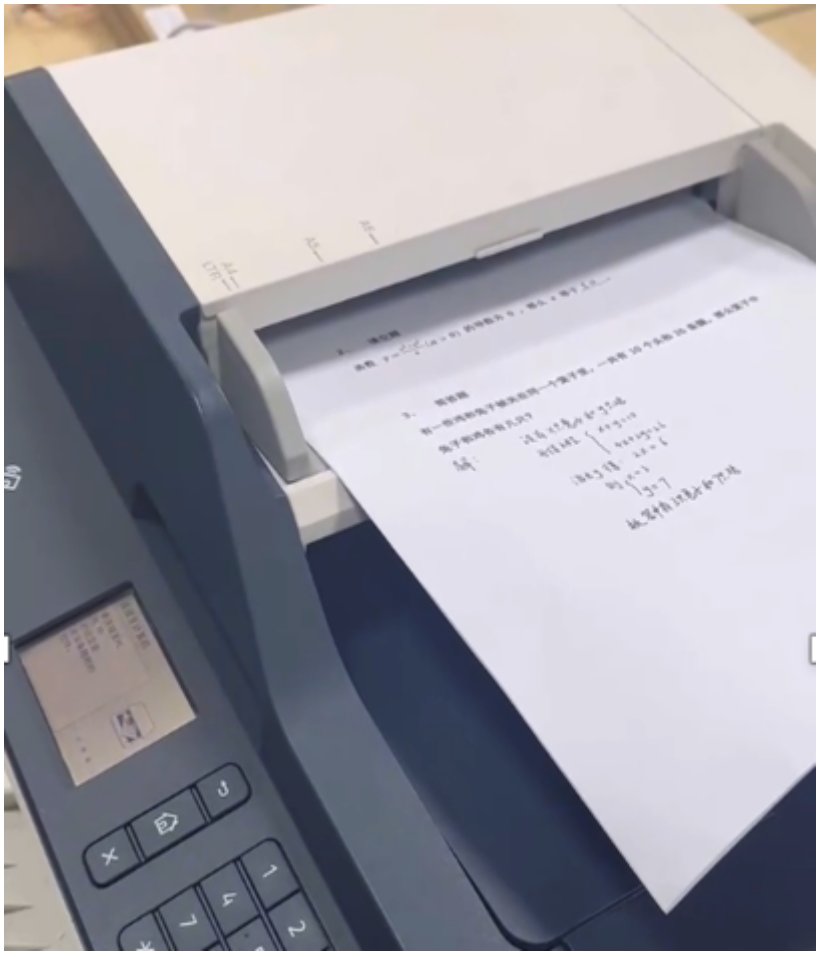
While existing conventional products on the market, such as Zuoyebang and Xiaoyuan Souti, can successfully recognize and judge questions, they still require users to manually correct, write comments, and grade them. This means that these conventional products fail to truly alleviating the workload of teachers, who still need to invest significant time and effort in grading

assignments, especially when dealing with bulk assignments and exams, where manual grading is labor-intensive, inefficient, and prone to errors.

To address this issue, we have customized specific algorithms in the software part, leveraging the interfaces and plugins of conventional printers to enable them to automatically perform traceable grading tasks. Specifically, we utilized the following interfaces and plugins:

- **Printer Driver Interface**
  Controls the basic functions of the printer, such as printing, scanning, and copying.
  - **Print Document Package API**: Provides interfaces that allow applications to access and manage print document packages, which is necessary for initiating print tasks in this project.
  - **Print Spooler API**: Provides interfaces for the print spooler so that the cloud can manage printers and print jobs. In this project, since exam papers are inserted in order, it is necessary to arrange the printing sequence of grading results reasonably.
  - **Network Interface**: Allows the printer to connect to the local area network, upload scanned results, download and print graded results via the network.
- **Image Processing Plugin**
  Processes scanned image data, including image compression, denoising and enhancement.
- **Encryption Plugin**
  Uses encryption algorithms to encrypt the uploaded and downloaded data, ensuring the security of data transmission.

The actual product is shown in the figure below:



In this project, users only need to place the papers to be graded into the paper tray of the hardware device, and the device will automatically complete the following tasks:

- **Scanning**: The hardware device uses a built-in high-precision image sensor to scan the students' paper assignments. This sensor can accurately scan handwritten text and symbols.
- **Data Processing and Upload**: The scanned results are are compressed and encrypted using AES before being uploaded via the device's built-in Wi-Fi communication module. The data upload process uses HTTP POST requests and interacts with the cloud server through RESTful API interfaces. The cloud decrypts and grades the received encrypted data.
- **Result Download and Printing**: The grading results are downloaded to the local printing system through an encrypted communication port. Depending on the user's choice, the system offers two printing methods:
    - **Directly Printing Grading Results on the Original Paper**: The device directly marks the grading results and comments on the original paper. The built-in position calibration module ensures the accuracy of the printing position.
        - **Printing the Paper and Grading Results on Blank Sheets**: The cloud integrates the scanned paper content with the grading results, then prints them on new blank sheets. This method accommodates specific requests from teachers, such as preserving the original paper.

These technical means and specialized hardware effectively reduces the time and effort teachers spend on manual grading, truly achieving the goal of alleviating teachers' workloads.

Project Advantages

1. **Efficient Grading Process**: This project introduces an integrated automatic scanning and error correction system. By adopting numerous user request acceleration and high concurrency technologies, such as Redis caching and parallel processing methods based on JAVA multithreading technology, and by adjusting hyperparameters to control model response time, speed is maximized. It can process over 100 papers per minute, significantly speeding up the grading process. Compared to traditional manual grading methods, this efficiency greatly reduces the time and effort required by teachers.
2. **Intelligent and Accurate Analysis**: The system uses advanced large model technologies for intelligent grading. The model not only completes accuracy statistics and score calculations but also identifies and records specific errors. Through error tagging and location statistics, it deeply analyzes each student's knowledge gaps and generates detailed feedback reports, including error type distribution and knowledge point mastery. This functionality helps teachers better understand students' learning situations and enables the project to provide personalized error notebooks based on the error analysis results for targeted review and improvement.
3. **High Accuracy and Low Cost**: The model employs a specially designed LLM adversarial interaction system, integrating RAG and MoE-related technologies to dynamically adjust and optimize model parameters, supplemented by user feedback. This system achieves a grading accuracy rate of over 97% and can grade questions that general question-solving software cannot recognize, such as drawing questions and questions with images. By combining image recognition and natural language processing technologies, it overcomes the limitations of traditional OCR and text recognition methods. Additionally, the processing cost per paper is only 0.05 yuan, offering significant cost advantages and providing a highly cost-effective solution for educational institutions.
4. **Excellent User Experience**: This project innovatively adopts a combination of software and hardware capable of utilizing existing printers for traceable grading results printing. On the one hand, the printer automatically performs traceable grading, further reducing teachers' workloads. On the other hand, the system is easy to operate, with teachers only needing to place the papers in the tray and press a button to receive the graded papers in a short time.

Additionally, the project supports customizable formats, allowing the grading format to be flexibly adjusted according to the regulations of different schools and teachers. This excellent user experience has received high recognition and positive feedback from teachers.

Commercialization of the Project

This project fully considers the needs and characteristics of the education market, achieving diversified revenue sources through both To B and To C business models.

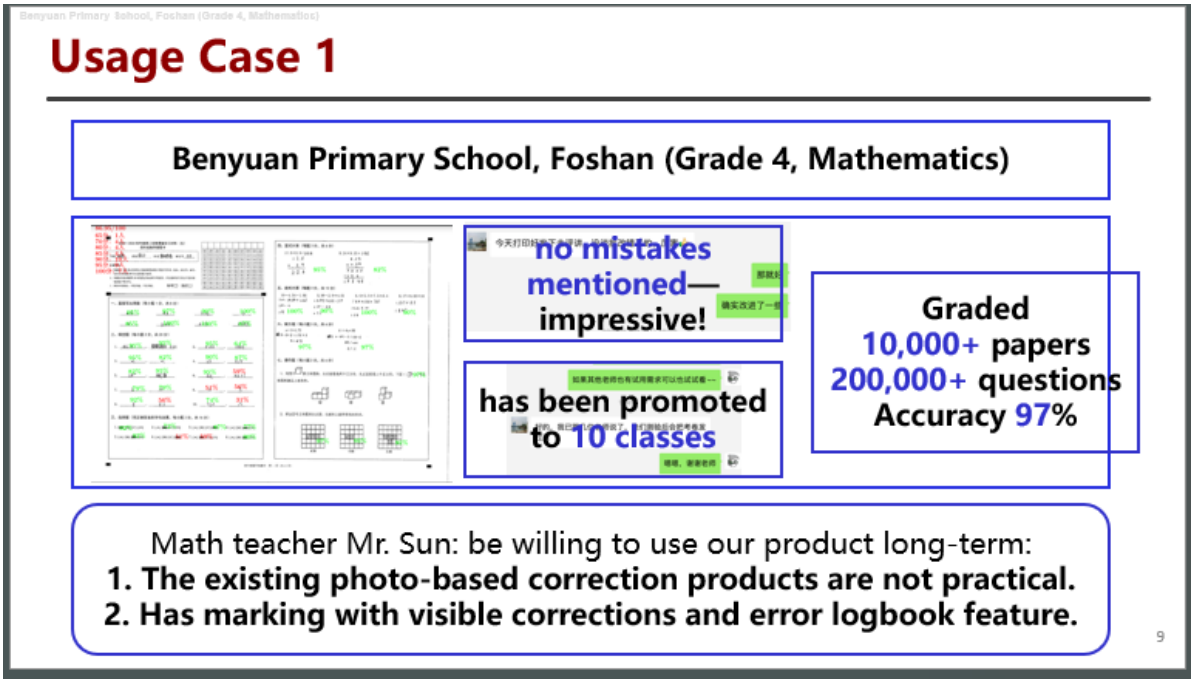
In the To B model, the primary customers are various educational institutions and schools. The project offers comprehensive solutions for information equipment and intelligent correction systems, including hardware procurement and software deployment. Schools can purchase services as needed, ensuring continuous access to the latest technical support and maintenance services.

The To C model directly targets student parents, offering personalized education services. Parents can purchase error notebooks and targeted tutoring materials generated by the intelligent grading system for their children. Additionally, we provide a series of extended services, such as online tutoring, learning progress tracking, and personalized learning suggestions.

Under the background of increased national investment in education and rapid advancement of informatization, the education market in China has a scale of over 600 billion RMB, affecting 400 million people, with informatization investment accounting for 15%. This provides a broad market foundation and development potential for the main body of this project, namely the intelligent grading system and related services.

Usage Cases

Benyuan Primary School, Foshan (Grade 4, Mathematics)



Wensan Street Primary School, Hangzhou (Grade 3, English)

Usage Case 2

Wensan Street Primary School, Hangzhou (Grade 3, English)

very satisfied with this version.

Graded
10,000+ papers
200,000+ questions
Accuracy 97%

English teacher Ms. Hu: be willing to pay for our product:

1. **The scanner previously purchased has been left unused**
2. **She is willing to pay for our product and to further promote it.**